sPHENIX
Global Level 1 / Global Timing Module

Firmware version 51

11-11-23

1.0 Overview

The operation of the DAQ system is governed by the Global Level-1 Trigger (GL1) and Global Timing System (GTM), which instructs the front-end electronics to “select” (or accept) the data from a given collision, or not. If accepted, the data sent up from the front-end and are eventually stored on disk and tape. This operation is commonly referred to as “triggering”. The GL1 decision to accept the data from a given collision is based on the input from a number of Local Level-1 systems (LL1), which examine the data from various detector systems and communicate a number of key properties to the GL1. After taking the various LL1 inputs, the overall “busy” state of the DAQ system, and several other factors into account, the GL1 either accepts or rejects the data of the collision in question. It takes this decision for every beam crossing. Once a collision is accepted, the GL1 instructs the Timing System to inform the front end detector subsystems of this fact. The timing system then sends this information to the front-ends. (sPHENIX TDR). The overall system block diagram with I/O interfaces between the LL1, GL1 and GTM systems are shown below in figure 1.

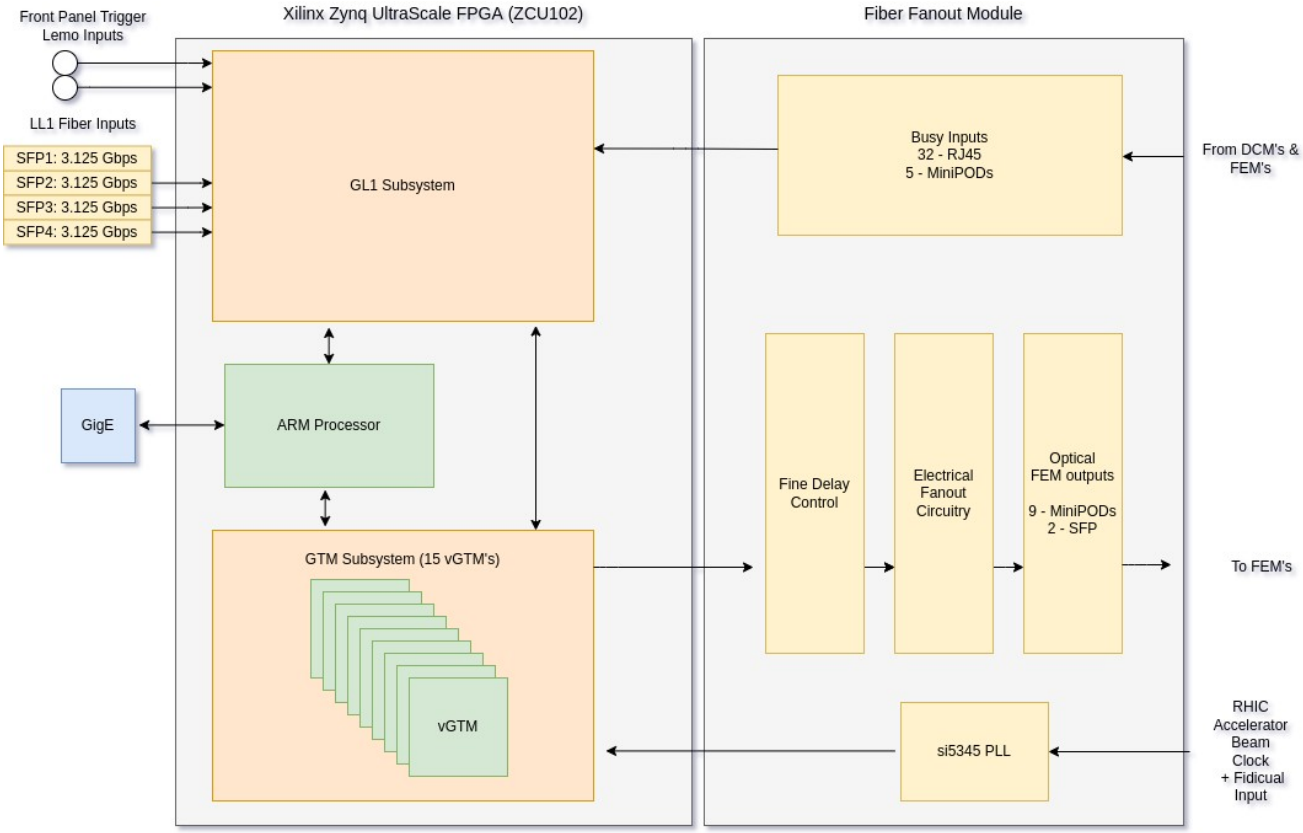

figure 1. GL1-GTM block diagram

2.0 Hardware

The GL1-GTM system currently uses a Xilinx ZCU102 Evaluation Board in combination with a custom Fiber Fanout Board. It is housed in a standard 3U rack mountable enclosure along with a Front Panel LED status board and power supply.

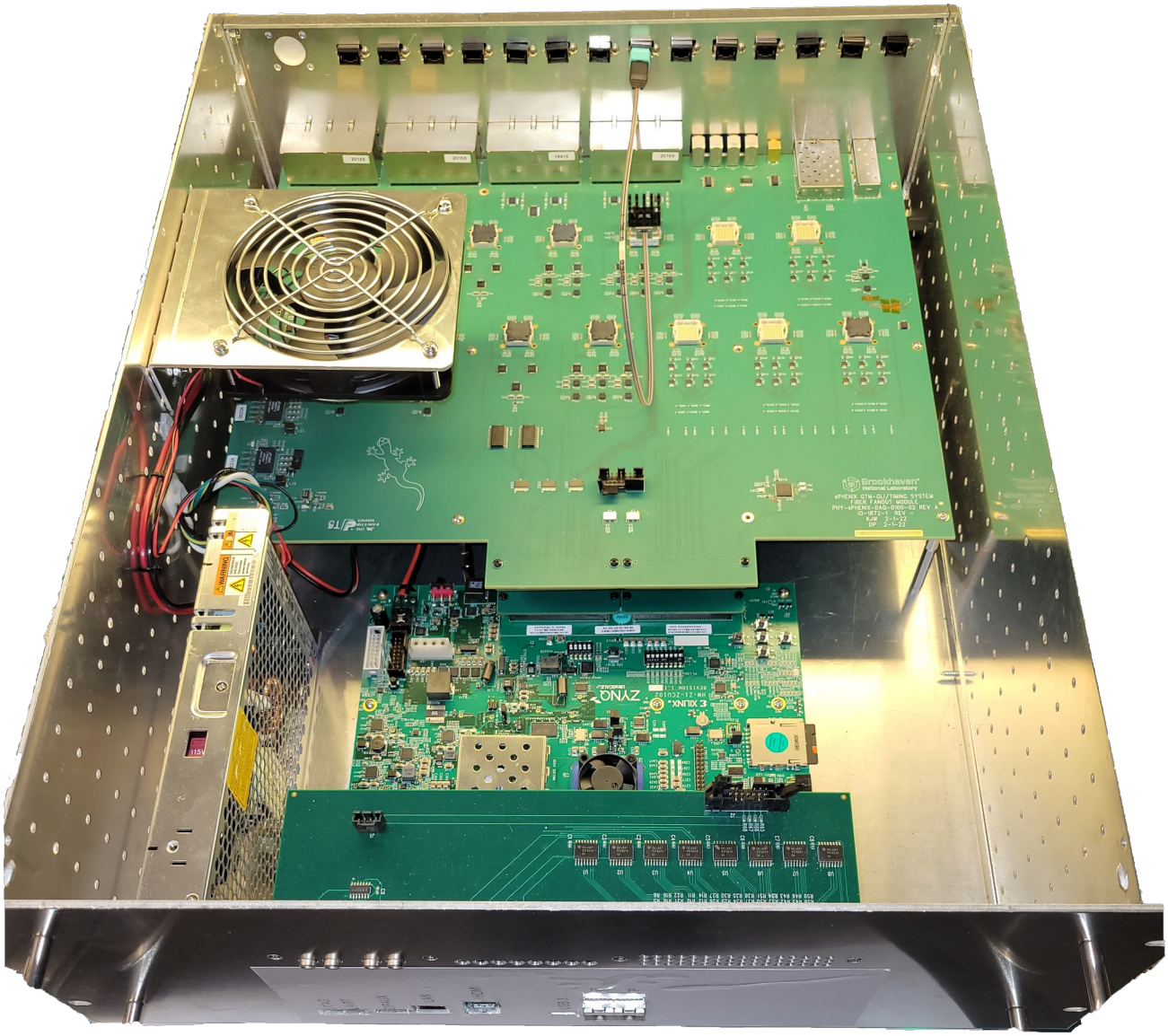

Figure 2. GL1-GTM Hardware

2.1 FPGA board

The FPGA board is a standard Xilinx Evaluation board (ZCU102) based on the Zynq® UltraScale+ ™ XCZU9EG-2FFVB1156E MPSoC (multiprocessor system-on-chip). High speed DDR4 SODIMM and component memory interfaces, FMC expansion ports, 24 multi-gigabit per second serial transceivers, a variety of peripheral interfaces.
The SDCARD memory provides non-volatile memory for storing the fpga bit file, linux kernel and root file system. Since the Zynq UltraScale+ FPGA includes a hard-core A9 processor, the system runs Debian Linux, which makes communicating to the device the same as connecting to a standard linux computer, a user can log into the system via ethernet using the ssh protocol. A serial port connector is also provided for a console login. This board also provides over 120 signal connections as well as 16 high speed transceiver lanes via the 2 FMC connectors which mates to a custom Fiber Fanout Module.

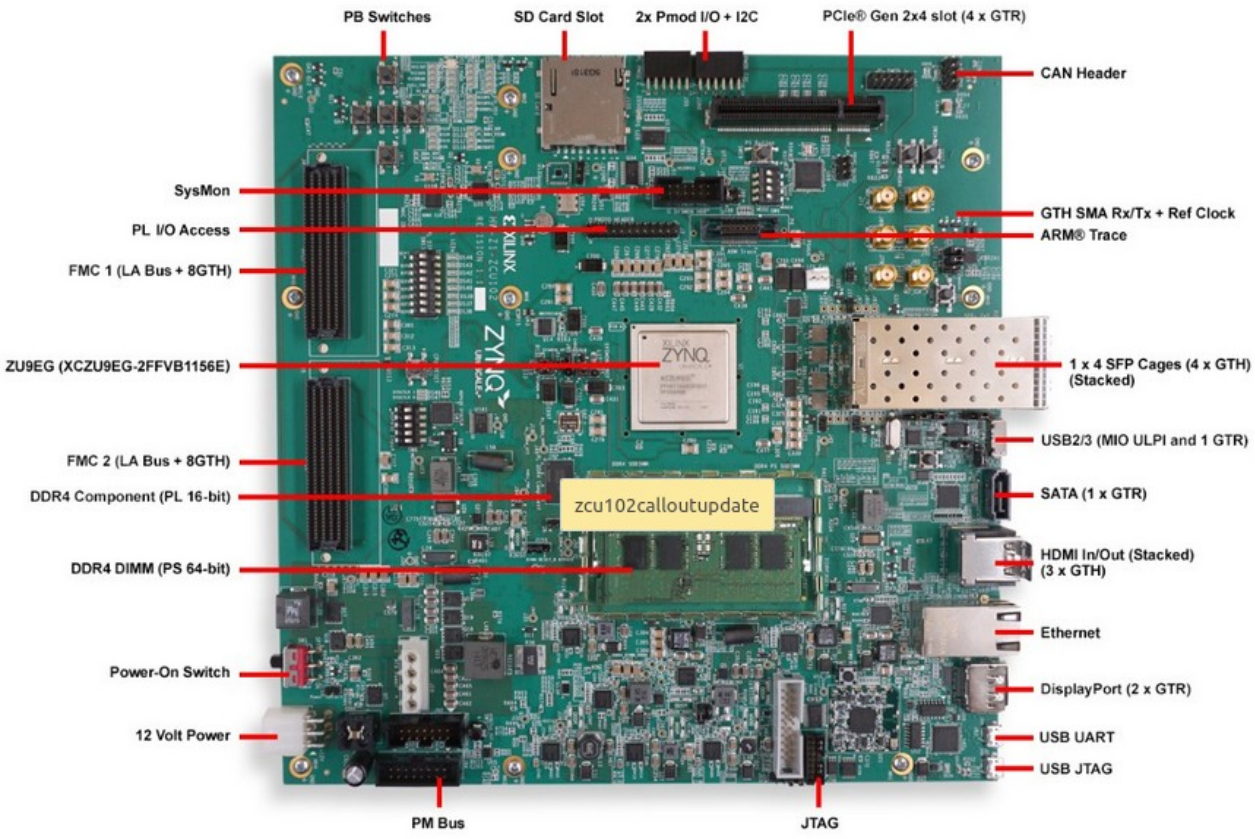

Figure 3. Xilinx ZCU102 Evaluation Board

2.2 Fiber Fanout Board

The transition board at the back of the enclosure provides the expansion capability for the system, as it has over 80 differential connections available to/from the FPGA board, as well as 8 high speed transceiver lanes. The board is currently under development and includes all the electrical to optical converters for fiber outputs to the various detector subsystems front end modules, the RHIC accelerator clock input, RJ45 connectors for DCM busy inputs and also generic I/O via lemo connectors.

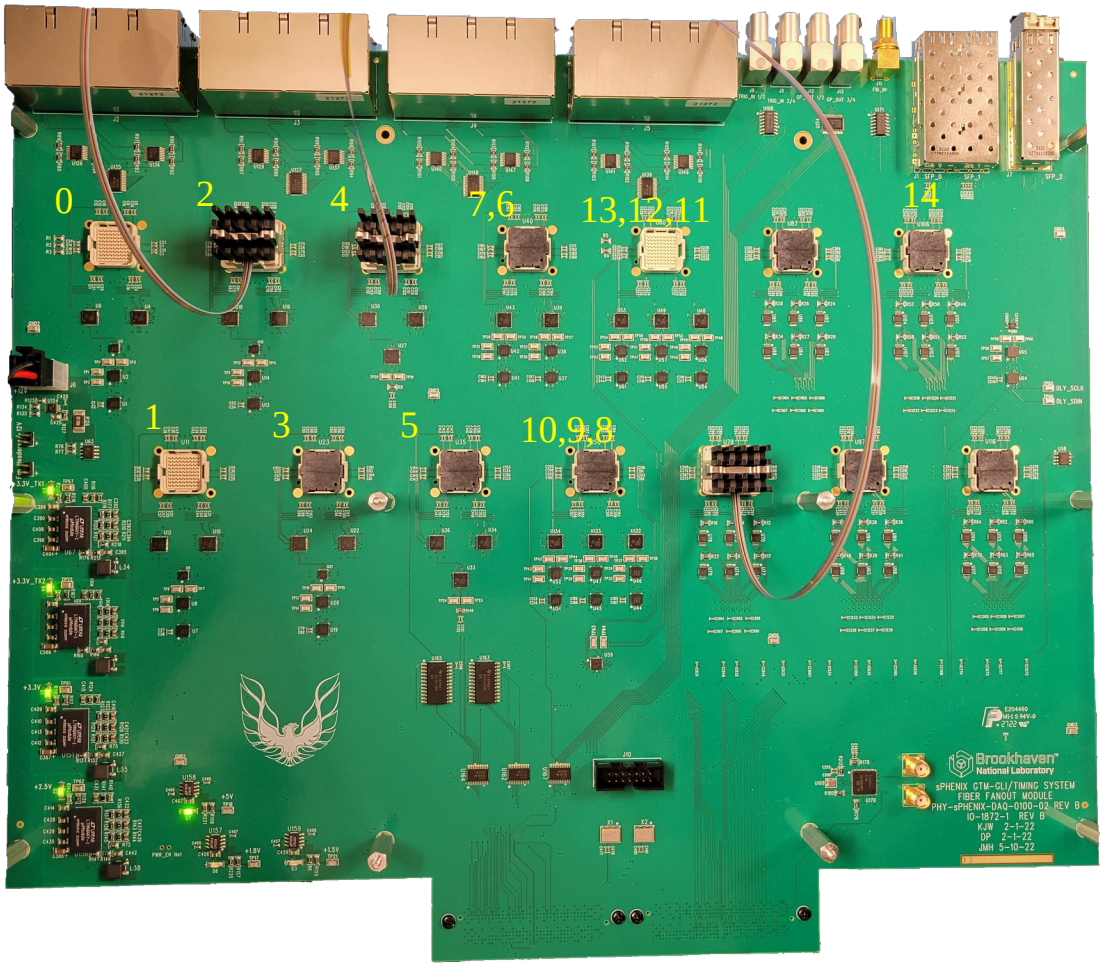

Figure 4. Fiber Fanout Module

3.0 Granule and FEM Fiber Count

The GL1-GTM module provides the distribution of the timing information to the front end modules for the various subsystems which are partitioned into granules. The independent operation of these granules is necessary so that different subsystems can be run independently with their own unique mode bits. This is supported by the GTM subsystem by the instantiation of multiple virtual GTM blocks within the FPGA, each of which require their own dedicated transceiver. The limitation of the number of available granules for sPHENIX is therefore dictated by the number of transceivers supported by the hardware. The existing hardware platform supports 14 transceivers, 4 of which are used for the LL1 and JSEB interface, leaving the possibility of up to 10 transceivers to be used for supporting different granules. The table below shows the current wish list for front end granules.

VGTM#	Detector	Fine Delay	Fanout	MiniPOD #
0	TPC East	Yes	1x12	0
1	TPC West	Yes	1x12	1
2	INTT	Yes	1x12	2
3	Spare	Yes	1x12	3
4	MVTX	No	1x12	4
5	Spare	No	1x12	5
6	EmCAL	Yes	1x6	6
7	EmCAL	Yes	1x6	6
8	OHCal	Yes	1x4	7
9	IHCal	Yes	1x4	7
10	MBD	Yes	1x4	7
11	TOT	Yes	1x4	8
12	EPD	Yes	1x4	8
13	Spare	Yes	1x4	8
14	Spare	Yes	1x1	SFP

Table 1. vGTM to Fiber mapping

4.0 RHIC Accelerator Interface

The exact details of this interface are still be worked out but the end result will be obtaining a 6 times beam crossing clock (~56.299MHz) from the accelerator along with a fiducial marker which occurs at the revolution rate. sPHENIX must remain locked to this clock during a run, although it is not guaranteed that it will stay locked during acceleration as the beam crossing clock frequency changes during this time, the magnitude of change is dependent on the species of ion's in the accelerator.

4.1 Beam Crossing Counter

The global 64-bit beam crossing counter is maintained in the GL1-GTM system and its function is to provide a timestamp for all data generated by the various subsystems. Upon a GL1 trigger, the count value is latched and the lower 40 bits are transmitted to the GTM subsystem where it is then forwarded to the front end systems. The latched value is also attached to the packet going to the jSEB interface which also includes all the GL1 counter values.

The beam crossing counter needs to be pseudo non-volatile in that if for some reason the GL1 power is lost or the system is rebooted the count value is not lost. The exact details of how this is done have not been entirely decided. One possible option could be to break it into 2 – 32 parts. The upper 32 bits is synchronized to a time server and provides the epoch time. The lower 32 bits is a count value of the beam crossing clock, which is reset to zero whenever at 1Hz, when the epoch time is updated. This is currently the method used by the NSLS-II Timing System.

4.2 Global Clock Select

The GTM-GL1 system runs at 6 times the RHIC accelerator beam crossing clock, which is nominally 56.4MHz. The input RHIC clock directly feeds into an SI5345 PLL on the fiber fanout module. This PLL produces a frequency of 12 times the RHIC clock frequency which is used as the GTH reference clock inputs. This permits the transmission of 8B/10B data to the front end systems at a rate of 6 times the RHIC beam clock. The TXOUTCLK from the GTX transceivers is then used as the system clock for the system, which runs at 6 times the RHIC clock. There are times when the RHIC clock is unavailable, such as when testing in the lab. For these cases, the PLL will automatically generate a clock at the nominal 56.4MHz, so the system will still run at the same 6 times the RHIC clock in this situation. A fidicual pulse is also received by the system, which will indicate when the RHIC bunch number 0 is at a predetermined location. From the fiducial pulse, the phase 0 location will be determined. A fidicual selection is also provided for cases when the RHIC fiducial tick is not available. For this, an internal 120 count counter is provided to generate the fiducial tick.

5.0 Global Level 1 Subsystem

The top level block diagram for the GL1 Logic is shown below in figure 3. To successfully complete its task of issuing GL1 triggers with a minimal and deterministic amount of time, the logic must carefully communicate with the various other subsystems, which include the RHIC Accelerator, LL1, GTM, and jSEB. The interfaces to all these other subsystems are all via fiber connections through either the front panel SFP slots or SFP slots on the rear transition board. The communication links use the standard Xilinx GTX transceivers technology with 8B/10B encoding and run at various frequencies. All slow controls communication with the unit is via gigabit ethernet through the ARM processor, which runs Debian Linux and uses standard network socket communication. It is possible to ssh directly into the unit to control it locally.

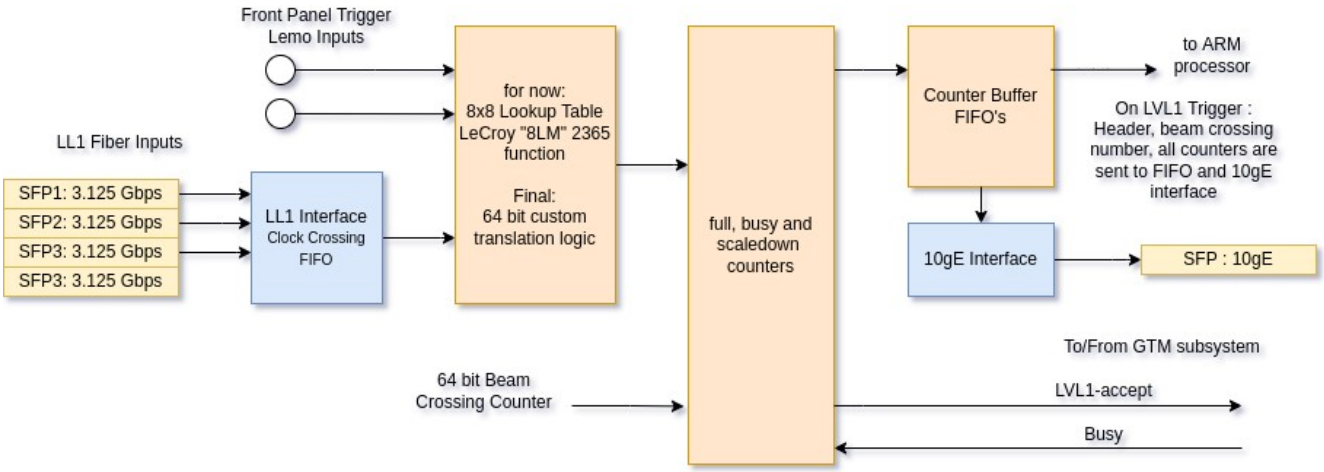

figure 3. GL1 Subsystem Block Diagram

5.1 Local Level 1 Interface

The Local Level 1 Interface receives the trigger information from the LL1 system which is the main input for Global Level 1 decisions. The interface will be via 4 - 3.125 Gbps 8b/10b links, the details are still being decided.

5.2 Trigger Logic

The current implementation accepts 8 trigger inputs which are fed into a lookup table to generate 8 LV1 triggers outputs. This provides functionality similar to what veterans remember as the LeCroy "8LM" 2365 unit, an 8-fold logic matrix that can be soft-configured (M. Purschke, DAQ meeting 3-25-20). The implementation is done using a dual port ram. Port A is used to configure the memory which is loaded from the ARM processor and runs at the system clock frequency. Port B is used to generate

the 8 LVL1 trigger outputs from the 8 trigger inputs and runs at the beam crossing clock frequency. The 8 trigger inputs form the address into the 256 byte lookup table.

Since the current hardware only has 2 front panel inputs, 8 additional trigger inputs are provided via soft triggers from the ARM processor to aid in testing. They form the input address to the lookup table as follows. A soft trigger is initiated by writing to the SOFT_TRIG register.

Trigger Input	Source
0	Front Panel(0) or Soft_Trig(0)
1	Front Panel(1) or Soft_Trig(1)
2	Soft_Trig(2)
3	Soft_Trig(3)
4	Soft_Trig(4)
5	Soft_Trig(5)
6	Soft_Trig(6)
7	Soft_Trig(7)

Table 1. Trigger Mapping

To write the lookup table, use the TRIG_LUT register. Bit 7-0 are used as the data input and Bits 15-8 are the address. For example to write the value 0x1 to address 0x2 of the table the following command would be issued.
> gl1_rw W 16 0x21 where 0x10 is the register address of the TRIG_LUT register

A helper program is provided which writes the entire 256 word lookup table which uses a text file input having 2 columns for the address and data.
> load_triglut <trig_file>

Once the lookup table has been loaded with appropriate values, soft triggers can be issued from the console to test the results. To write a soft trigger use the SOFT_LVL1_TRIG register. A helper program is provided to make it easier
> gen_ softlvl1 <trigger vector> where trigger vector are the 8 trigger inputs to the lookup table as shown in table 1.

5.3 Raw, Live and Scaledown Counters

The GL1 counters provide a running snapshot and control of the sPHENIX trigger rates. Even though the current number of trigger outputs are limited to 8, the counter section instantiates counters to accept the final number of 64 triggers. The outputs of the lookup table are fed into the counters where each of the outputs are connected to 3 different counters. The raw counter will increment on every trigger, the live counter will update only when the busy signal is inactive and the scaledown counter will increment when the number of triggers reaches the scaledown factor. Each of the 192 counters is 64 bits. Trigger 0 is a special case in that it counts beam crossing clocks and is not tied to

any output from the Trigger LUT. It is important to write an appropriate scaledown value for this trigger, otherwise GL1 triggers will occur on every beam clock.

Trigger LUT output bit	Source
0	Not connected. Trig 0 Counters are connected to beam clock
1	Trig 1 Counters
2	Trig 2 Counters
3	Trig 3 Counters
4	Trig 4 Counters
5	Trig 5 Counters
6	Trig 6 Counters
7	Trig 7 Counters

Table 2. Trigger LUT output bit to Counter Mapping

The first consideration is to load the scaledown factors for each of the 64 counters. By default, they are programmed to 0 which will provide no scaledown. Writing a 1 will give a scaledown of 2, writing a 2 will give a scaledown of 3 and so writing a value of n will give a scaledown of n+1. If a value of 0xFFFFFFFF is written to the scaledown, then that scaledown trigger is disabled. It is important to write an appropriate scaledown value for Counter 0, which is tied to the beam crossing counter, otherwise GL1 triggers will occur on every beam clock.

A helper program is provided to load the scaledown factors for each of the 64 triggers which reads a text file with 64 rows corresponding to the 64 scaledown factors.
> load_scaledowns <scaledown file>

A global counter reset is provided which will reset all counters to zero.
> ./gl1_rw W 24 1 -- puts counters in reset
> ./gl1_rw W 24 0 -- removes reset

A counter enable is provided for each trigger which can be used to mask out particular triggers. Since there are 64 enable bits and the register width is only 32 bits, it requires 2 writes to update all the enables. To keep the enables synchronized so that they are all enabled at the same time, it is only after writing the upper 32 enables register that they are all latched together. To enable the registers write to the COUNT_ENB_LO and COUNT_ENB_HI registers. Counters are disabled (masked) by default.
> ./gl1_rw W 20 0xFF -- enable counters 7-0
> ./gl1_rw W 21 0x0 -- disable counters 63-32

The counters are now be in a ready state, waiting for trigger inputs. A resulting GL1 trigger is asserted if any of the 64 scaled counters increment and sent to GTM's for transmission to the front end

detectors. For each GL1 trigger generated a snapshot of all 192 counters are clocked into FIFO's along with the value of the beam crossing counter so that they can be transmitted to the jSEB interface for transmission to the jSEB for archival. A throttled version of the same information is also available to the ARM processor for slow feedback of the state of the counters. To read out the counter values from the ARM processor a helper program is provided.

> get_countstats

A global counter is also provided, register CNT, which provides a running count of the GL1 triggers asserted from any of 64 scaledown triggers. This counter is cleared with the same CNT_RST register that also clears all the counters.

5.4 10 Gigabit Ethernet Interface

The 10 gigabit ethernet interface provides communication of all trigger count values (raw,live,scaledown) as well as other trigger related information from the GL1. This communication protocol is UDP and is transmitted via the SMA GTY outputs on the ZCU102 board. The bulk of the total data rate transmitted are the three scalers (raw,live,scaledown) for each of the 64 possible trigger conditions. This results in 64*3*8bytes = 1536 bytes, plus 54 bytes for general information, which comes to 1664 bytes total for each trigger. Each packet is zero padded to make it 2000 bytes. With a maximum throughput rate of 15KHz the average data rate over the fiber will be 25Mbytes/sec. The format for the data payload via gigabit ethernet is shown in table 3 and 4. The data is transmitted via a single jumbo frame as shown below in table 3. Data is pushed at each GL1 trigger.

64 bit word Number	Function	Value
0	Header	0xface (16 bits)
1	Packet Number	Packet Number (32 bits)
2	Beam Counter	Beam Crossing Counter at GL1 trigger
3	Bunch Number	Bunch Number at GL1 trigger,
4	Reserved	0x0123456789abcdef
5	Trigger Vector	Input Trigger Vector
6	Trig LUT Output	Trigger LUT Output Vector
7	Reserved	0xdeadbeefba5eba11
8	Trig0:Raw	Raw Counter for Trigger 0
9	Trig0:Live	Live Counter for Trigger 0
10	Trig0:Scaled	Scaled Counter for Trigger 0
11	Trig1:Raw	Raw Counter for Trigger 1
12	Trig1:Live	Live Counter for Trigger 1
13	Trig1:Scaled	Scaled Counter for Trigger 1
...	...	
189	Trig63:Raw	Raw Counter for Trigger 63
190	Trig63:Live	Live Counter for Trigger 63
199	Trig63:Scaled	Scaled Counter for Trigger 63
200	Trailer	0xdecafbad
201	Filler	Zero Padding to get 2000 bytes.

Table 3. UDP data packet

5.5 10 Gigabit Ethernet Busy

Each packet that is transmitted must receive an acknowledge packet, which ensures the DAQ system has received and processed the information. This requirement is enforced by the GL1_DAQ_BUSY_ENB register. If the GL1_DAQ_BUSY_ENB register is 0, then acknowledgement packets will be ignored and the GL1 system will never go busy due to the DAQ system.

The response packet from the DAQ system must contain three 32 bit words with the following structure

Word Num	Value	Description
0	Key	0xdeadbeef
1	address	1
2	Value	Reserved (ignored for now)

When the response packet is received, the state machine that controls reading out the events trigger counters FIFO will stop waiting for the acknowledge packet and permit the next event trigger counters to be transmitted out the 10gE link. If no response packet is received for 16 transmitted events, the GL1 subsystem will go busy. The readback register GL1_DAQ_BUSY_STAT provides status information on the state of the DAQ busy.

6.0 Granule Timing Module Subsystem

The sPHENIX granule timing system provides distribution of the timing information to the front end modules (FEM) for the various detector subsystems. The timing module performs three primary functions. First, it is a mode bit scheduler, which outputs predetermined mode commands on a clock by clock basis to the FEM's. Second, it provides readout enable strobes to the FEM's, and monitors the number of level-1-accepts generated by the level1 system. Third, it provides low jitter distribution and generation of timing signals, namely the Beam Clock, Beam ClockX6 (generated on board via PLL), and LVL-1 Accept.

6.1 Mode Bit Scheduler

The timing module provides an eight bit encoded command word (mode bits) for each beam crossing to all FEM's. Mode commands are defined as the encoded value of the eight mode bits. Mode commands are grouped together, into mode command groups where a group represents a single revolution of the RHIC accelerator. At the current time there are 120 such beam crossings per revolution, therefore a mode command group consists of 120 mode commands. With the use of FPGA memory blocks as the storage devices for mode commands and a separate memory block containing a

list of pointers (scheduler commands) to the beginning of any Mode Command Group, mode bits can be defined for unlimited number of cycles (with repetition).

6.11 Scheduler Commands

Scheduler commands are 32 bit instructions loaded in the scheduler memory, which has a depth of 128. The scheduler commands are to be loaded in the order of planned execution. The format for the scheduler command is shown in table 2. Bit 31 is the retransmit bit. A logic 1 in this bit will reset the scheduler after its operation is complete, thereby looping back to the scheduler command address that is specified byt the LOOPBKADDR field. Using this loopback feature mode commands may be issued to the FEM's for an unlimited time and up to the first 8 mode command groups can be setup as an execute once group for initialization routines. Bits 30-8 define the Scheduler Command repeat bits. The binary value assigned here represents the number of times to cycle through the current mode command group. Bits 4-0 define which of the 32 unique mode command groups to perform. The binary value assigned here is a pointer into the command memory to a unique mode command group. The address of the currently executing scheduler command may be read from the Timing Control and Status Registers CYCLE_PTR.

Field	Bits	Function
RETRAN	31	Retransmit
CYCRPT	30:8	Scheduler Command Repeat Value
LOOPBKADDR	7:5	Mode Bit Group Number to loop back to when RETRAN is asserted
CYCCMD	4:0	Mode Bit Group Number

Table 4. Scheduler Command Bit Definitions

6.12 Mode Commands

Mode commands are 32 bit instructions loaded in the Command Memory, which has a depth of 4K. The first byte, (D7-D0) loaded into this memory are the mode bits which are destined to the FEM's via fiber. The upper bytes are reserved for future use. There is room for storing 32 unique mode command groups (each mode command group consists of 120, 32 bit, mode commands). The first mode command of each of the mode command groups must be aligned at 128 word offsets.

6.13 Mode Bit Scheduler Operation

A quick overview of the procedure for loading and running the scheduler is described. Mode bits are loaded into the command memory in groupings of 120, which is the number of crossings per revolution. Each new grouping starts on a 128 word boundary. Then the scheduling commands are loaded into the scheduler memory with the format described in table 2. Up to 128 scheduling commands may be loaded. When the mode bit scheduler starts, by a global start signal, the control logic fetches the first command from the scheduler memory. The lower five bits form the upper bits on

the command memory which is a pointer to the base address of a unique mode bit group. A cycle counter then begins which counts from 0 to 119 on each beamclk, which provides the lower bits on the command memory This counter then runs until the Cycle Repeat Field (CYCRPT) has been decremented to zero. The control logic block recognizes this and then fetches the next command from the scheduler memory and the process repeats itself. If the retransmit bit is set (RETRAN), then the next cycle command address will be the first memory location of the scheduler memory.

6.14 Editing Single and Cycle Commands

To modify and/or read any of the scheduler commands (scheduler memory) and/or mode commands (command memory) the module must be in a stopped mode. Attempting to change this values while the module is in run mode will have no effect and attempting to read these values while the module is in run mode will result in undefined data. The control and status registers are always available. The procedure for stopping the module is to set the *STOP_LOCAL* bit of the Timing control and status register and then poll on the *RUN* bit to verify that the module has stopped. (The module may not stop immediately, depending on the value of the bits *STOP_EOSF* and *STOP_EOCC* of the Timing control and status register) Once the module has been placed in the stopped mode both the scheduler commands and the mode commands may be read and/or modified. After any modifications have been performed the module may then be restarted either locally or globally by the methods described in the next section

6.15 Global/Local Start/Stop Procedures

The timing module is started and stopped through the global control inputs: global start and global stop. Two control bits, *START_ENABLE* and *STOP_ENABLE*, also exist which enable the modules to accept and process these global signals. These control bits allow groups of timing modules to be synchronously started and stopped. Upon receiving a global start pulse and having the control bit *START_ENABLE* asserted, the module enters a arming state where it then awaits a fiducial clock pulse, which synchronizes it to a predetermined RHIC accelerator bucket. The next beam clock pulse puts the module into a run mode and mode bits are output. To globally stop the module the board must receive a global stop pulse and have the control bit *STOP_ENABLE* asserted. Then depending on the current value of the control bits: *STOP_EOSF* (stop at end of sequence fifo) and *STOP_EOCC* (stop at end of current cycle (see section 4.2 for details on these bits)), the module will stop accordingly.

6.2 Level-1 Accept Handler and Readout Enable Generation

The timing module provides a mechanism for handling the LVL1-Accept pulses generated by the GL1 subsystem and providing readout enable pulses to the FEM's based on these pulses.

6.21 LVL1-Accept Handler

To prevent data overrun in the FEM's or in the DCM's, the rate of LV1-Accept triggers must be monitored. This is accomplished by tracking the current number of LVL1-Accepts that are queued for readout with a three bit counter *EVT_CNTR*. If this counter reaches a value of 5 then the busy signal is asserted and returned to the GL1 subsystem, thereby inhibiting it from generating any more LVL1-Accept pulses. Upon reaching this count value of MAX_EVENTQUEUE, a presettable longtime timer, *LTTIMER*, is started. This timer has a maximum time of about 6.5ms, but may be preset by the user for a shorter duration. When this timer reaches zero, the *EVT_CNTR* should also be at zero. If it is the busy signal is negated, enabling the GL1 system to resume sending LVL-Accepts. If it is not, then there is a data-flow problem (Most likely the *DCMBUSY input is asserted) and the error bit *CNTERR* is asserted.

6.22 Data Collection Module Busy

The Fiber Fanout module accepts 32 DCM_BUSY inputs is which are provided from the DCMs, and is used to prevent the FEMs from passing more data to the DCMs if they have signaled that they are full, can't unload, etc. This is accomplished in the logic described above by only letting the current data transmission (both ENDAT1 and ENDAT2) run to completion. No further LV1-Accepts are generated by GL1, and when the DCMs are no longer "busy", the above protocol resumes.

Each vGTM has a 32 bit mask register DCM_BUSY_MASK that is used to enable only desired input bits. After the logical and of the DCM_BUSY input and DCM_BUSY_MASK register, the vector is logical OR’d to create a single dcm_busy bit.

6.23 FELIX Busy

The Fiber Fanout module accepts 60 FELIX BUSY inputs is which are provided from the FELIX cards, and is used to prevent the FEMs from passing more data LVL1_ACCEPT pulses to the FEE’s if they have signaled that they are full, can't unload, etc. This is accomplished in the logic described above by only letting the current data transmission (both ENDAT1 and ENDAT2) run to completion. No further LV1-Accepts are generated by GL1, and when the FELIX cards are no longer "busy", the above protocol resumes.

Each vGTM has a 64 bit mask register FELIX_BUSY_MASK that is used to enable only desired input bits. After the logical and of the FELIX_BUSY input and FELIX_BUSY_MASK register, the vector is logical OR'd to create a single felix_busy bit.

6.24 Readout Enable Generation

With the use of the LVL1-Accept pulses, the timing module controls the readout of data from the FEM's to the DCM's with two signals, ENDAT0 and ENDAT1. These two signals are output on optical fiber to the FEM's. In general, when a LVL1-Accept is received the following occurs depending on the current value of EVT_CNTR.

LVL1-Accept received when EVT_CNTR = 1
The LVL1-Accept is passed to the FEMs via an optical fiber. Also a 16 bit programmable counter, *FEM_CNVRT_TIME* is started. This timer allows the front end to complete the conversion. The nominal time setting for this counter is 40uS. After this timer has timed out the first data enable (ENDAT0) is asserted. The width of this signal is the time required for the even FEMs to transmit their data and is controlled by a 16 bit programmable counter, *ENDAT_TIME*. After this timer has timed out, ENDAT0 is negated and ENDAT1 is asserted. The width of ENDAT1 is the time required for the odd FEMs to transmit their data and is controlled by the same counter, *ENDAT_TIME*. After this timer has timed out, ENDAT1 is negated and the counter EVT_CNTR is decremented.

LVL1-Accept received when EVT_CNTR > 1
The same as the case EVT_CNTR = 1, however the counter, FEM_CNVRT_TIME, will not be started until the beginning of ENDAT1 has started.

6.3 FEM Fiber Outputs

All outputs from the timing subsystem to the FEM's will be via optical fiber running at 6 times the RHIC bunch crossing clock frequency (9.4MHz x 6 = 56.4MHz) The data will be encoded in 8B/10B, and will be generated by the internal Xilinx Zynq GTX transceiver, so the line rate will be 1.128Gbit/sec. Therefore for each beam crossing, up to 6 values of 16 bits can be distributed. The primary instruction that is transmitted to each front-end unit for each RHIC crossing is 8 bits of information, historically called the mode bits, that convey the desired mode of operation of the front-end for that particular crossing. The 6 16-bit values transmitted per crossing take on different meanings depending on the transmission number. In this way, the system transmits the 8 mode bits on the "primary" clock edge that coincides with the edge of the actual beam clock. This particular clock edge is tagged by bit 8 being asserted high at that edge. On that same edge, the system also transmits the "Level-1 accept" information in bit 9. Two more bits, called "endat 0" and "endat 1", are transmitted and held through the 6 cycles. They historically delineated the data transmission window from a front-end and are unused in the TPC system, but are a convenient marker that shows an active data transmission for debugging purposes. They may be used again in other systems. When bit 12, which is essentially a copy of bit 8, is high, it denotes that the least significant 8 bits hold the mode bit pattern for this crossing. At the same time, the 3 most significant bits hold user bits, that can be used for several purposes. At the current time, no particular use for the user bits has been implemented. When bit 12 is 0, the 3 most significant bits count through the remaining 5 clocks, which transmit a total of 40

bits from the global beam crossing counter. As described previously, the front-end copies those bits to the headers, and also copies the 20 least significant bits to the front-end ASICs, which embed those into the data stream, for data alignment verification. The current redundancy of bit 12 with bit 8, which make each transmission deterministic, allows us to transmit the different per-clock information in a different order, if that should become necessary. The bit definition is defined below in table 4.

Bit Number	Function	Beam clock phases					
		0	1	2	3	4	5
7-0	Mode bits /BCO	Modebits bits 7-0	BCO bits 7-0	BCO bits 15-8	BCO bits 23-16	BCO bits 31-24	BCO bits 39-32
8	Beam clock phase0	1	0	0	0	0	0
9	LVL1 accept	X	0	0	0	0	0
10	Endat 0	X	X	X	X	X	X
11	Endat 1	X	X	X	X	X	X
12	Modebit enable	1	0	0	0	0	0
15-13	User bits	3 user bits	0	1	2	3	4

Table 5. FEM Data

7.0 FPGA Register Interface

The FPGA registers are accessible to Linux via a provided kernel driver or using a simple mmap command from userspace. The physical address of the registers and their definitions are shown below. There is a global memory space for registers that affect global parameters. Separate address spaces are provided for each of the instantiated vGTM specific registers.

Global Settings Register Map
Base Address 0xA0000000

Address	Subsystem	Register Name	Function
00	GTM	GLOBAL_START	Starts the scheduler for all vGTM's
01	GTM	GLOBAL_STOP	Stops the scheduler for all vGTM’s
02	ALL	FIDTICK_SEL	Selects if the fidicial tick is generated internally from a counter or from the SMA input on the FFO board. Bit 0 : 0 = internal, 1= external
03	ALL	FPGAVER	FPGA version
04	ALL	LED's	Bit 3:0 debug LED's
05	GTM	SCH_RESET	Resets scheduler (all vGTMs)
06	GL1	SOFT_LVL1_TRIG	Bits 7:0 : Generates a soft level 1 trigger to the trigger logic, useful for testing when external triggers are unavailable. Mapping for trigger inputs is shown in table 1.
07	GL1	SOFT_BUSY	Bit 0 : Generates a soft busy signal, useful for testing when external busy signals are unavailable.
08	ALL	GTM_GTH_RESET	GTM Transceiver Reset 0 = not in reset 255 = full reset
9	LL1	LL1_GTH_RESET	GTM Transceiver Reset 0 = not in reset 255 = full reset
10	ALL	FFO_TRIG_OUT	Trigger Outputs on FFO board
11	ALL	FFO_TRIG_IN	Trigger Inputs on FFO board
12	ALL	FFO_PWR_ENB	Power Enable on FFO board 0=FFO board powered off 1=FFO board powered on
13	ALL	FFO I2C RESET	Resets the i2c peripheral bus expander on the FFO board
14	ALL	RHIC_PLL_LOCK	Indicates if the GTM system is locked to the RHIC clock. GTM will output 56.299MHz when not locked. 0= not locked 1 = locked
15	GL1	GL1_ENABLE	GL1 triggers are disabled if the GTM scheduler is not currently running. Also, once the GTM starts, this register provides 2 bits to enable the GL1

			triggers from being processed. Bit 0 : GL1 Enable 0=disable, 1=enable Bit 1 : FA Enable 0=disable, 1=enable
16	GL1	LL1_ENABLE	Enables the LL1 fiber interface to connect to the 64bit GL1_trigger input 0=disable 1=enable
17	GTM	MAX_EVENT_QUEUE	Maximum number of events that can happen quickly before busy is asserted. 0 = 1 event 1 = 2 events ... 6 = 7 events (max)
18	GTM	GTM_BUSY	Readback of current vGTM busy state. Each bit is mapped to a vGTM. (Same information is on FP LEDs)
20	GL1	CNT_ENB_LO	Counter Enable Mask for Trigger Counters 31-0
21	GL1	CNT_ENB_HI	Counter Enable Mask for Trigger Counters 63-32 After writing this register, all 64 trigger counter enable registers are written simultaneously
23	GL1	CNT	Running counter of all GL1 events issued. Reset with the CNT_RESET register.
24	GL1	CNT_RESET	Resets the counter values for all the raw, live and scale counts.
25	GL1	LL1_CNTRL	Local Level 1 Data receive control bit0: FIFO Reset bit1: FIFO Enable
26	ALL	DCM_BUSY_IN	Readback of 32 bit RJ45 DCM busy inputs
28	GL1	GL1_PIPE_RST	Resets the pipeline delay interface. Should be momentarily asserted high on startup. Is reset on powerup.
29	GL1	GL1_PIPE_DLY	The GL1 counters that are read out by the 10gE interface are pipeline delayed by this number of beam crossings.
30	GL1	DEAD4N	Asserts GL1 busy for N beam crossings after a GL1 trigger
31	GL1	FA_MASK	Forced Accept Mask, each bit maps to a vGTM.
32	GL1	CNT_FIFO_DATA	FIFO Data for Count Values
33	GL1	CNT_FIFO_RDCNT	FIFO number of words available to read
34	GL1	CNT_FIFO_RESET	FIFO reset. Resets the FIFO’s for the GL1

			readout. bit 0 : (0=normal operation, 1=reset)
36	GL1	GIGE_TEST_ENB	Enables testmode data to be transmitted via the SFP5 slot bit 0 : 0=normal, 1=testmode
37	GL1	GIGE_TEST_TRIG	Triggers the start of a test packet to the JSEB bit 1 : 1=trigger Must be manually cleared to 0 before re-triggering
38	GL1	GIGE_TEST_STARTVAL	Bits 15-0 : The initial value to be transmitted in the test packet. Subsequent values will be incremented by 1
39	GL1	GIGE_TEST_LENGTH	Bits 31-0 : The number of 16 bit words that will be transmitted in the test packet.
40	GL1	GIGE_SRC_IP_ADDR	IP address for GTM-GL1 10gE interface
41	GL1	GIGE_RATE_CONTROL	UDP data rate limiter. Inserts a pause of n clock cycles between packets. Set to 0 for max speed.
42	GL1	GIGE_TEST_DATA	A counting pattern will be transmitted instead of GL1 counters. Bit 0 = 0: normal GL1 data Bit 0 = 1: test data
43	GL1	GIGE_RESET	Resets the 10gE MAC, should be done on powerup 0= normal operation 1=reset
44	GL1	GIGE_DEST_IP_ADDR	IP address that the GTM-GL1 sends the payload to
45	GL1	GIGE_MAC_ADDR_LOW	Lower 32 bits of GTM-GL1 MAC address
46	GL1	GIGE_MAC_ADDR_HI	Upper 16 bits of GTM-GL1 MAC address
48	GL1	GIGE_STATUS	Bit 16: Auto-Negotiation complete Bit 15: gige phy status
49	GL1	GIGE_PACKETS_RCVD	Number of gigabit ethernet packets received
50	GL1	GIGE_PACKETS_SENT	Number of gigabit ethernet packets transmitted
56	GL1	BC_CNT_LO	Beam Crossing Counter bits 31:0.
57	GL1	BC_CNT_HI	Beam Crossing Counter bits 63:32 After writing this register, all 64 beam crossing counter bits are written simultaneously
58	GL1	GL1_DAQ_BUSY_ENB	Enables the requirement that the GL1 DAQ system responds to each Gl1 transmitted event, otherwise system will go busy 0=gl1 daq busy disabled 1 = gl1 daq busy enabled
59	GL1	GL1_DAQ_BUSY_STAT	Bit 0: GL1 system is busy due to GL1 DAQ

			system. Bit 1: GL1 DAQ is waiting for a response packet.
60	GL1	SPIN_BC_CNT_SEL	Selects which of the 120 spin counters to select for readout
0x40-0x7F	GL1	SCALEDOWNS	Scaledown values for GL1 scaledown counters
0x80-0xBF	GL1	LL1_TRIG_DELAYS	Delay for each of the 64 trigger inputs. Range is 0-31. Step size is Beamclk:q
0xC0-0xFF	GL1	SWITCHYARD	Switch yard registers
0x100-0x1FF	GL1	TRIG LUT 0	Trigger Look up Table
0x200-0x2FF	GL1	TRIG LUT 1	Trigger Look up Table
0x300-0x3FF	GL1	TRIG LUT 2	Trigger Look up Table
0x400-0x4FF	GL1	TRIG LUT 3	Trigger Look up Table
0x500-0x5FF	GL1	TRIG LUT 4	Trigger Look up Table
0x600-0x6FF	GL1	TRIG LUT 5	Trigger Look up Table
0x700-0x7FF	GL1	TRIG LUT 6	Trigger Look up Table
0x800-0x8FF	GL1	TRIG LUT 7	Trigger Look up Table
0x2000-0x203F	GL1	GL1 Raw Counters	Raw counters for each of the 64 GL1 triggers (lower 32 bits)
0x2040-0x207F	GL1	GL1 Live Counters	Raw counters for each of the 64 GL1 triggers (lower 32 bits)
0x2080-0x20BF	GL1	GL1 Scaled Counters	Raw counters for each of the 64 GL1 triggers (lower 32 bits)
0x2100-0x213F	GL1	GL1 Raw Counters	Raw counters for each of the 64 GL1 triggers (upper 32 bits)
0x2140-0x217F	GL1	GL1 Live Counters	Raw counters for each of the 64 GL1 triggers (upper 32 bits)
0x2180-0x21BF	GL1	GL1 Scaled Counters	Raw counters for each of the 64 GL1 triggers (upper 32 bits)
0x2400-0x241F	GL1	GL1 Spin Raw Counters	Raw counters for each of the 16 spin triggers (the counters are from the beam crossing specified in the SPIN_BC_CNT_SEL register.
0x2440-0x245F	GL1	GL1 Spin Live Counters	Raw counters for each of the 16 spin triggers (the counters are from the beam crossing specified in the SPIN_BC_CNT_SEL register.

Virtual GTM Register Map
Base Address 0x43010000 + GTMnum * 0x10000
GTM0 Base Address : 0xA0010000
GTM1 Base Address : 0xA0020000
...
GTM13 Base Address : 0xA0140000

Address (hex)	Register Name	Function
00	SCHED_CONTROL	bit 0 : STOP_EOSF – Asserting this bit stops the scheduler as soon as the scheduler command with the retransmit bit asserted is completed. bit 1 : STOP_EOCC – Asserting this bit stops the scheduler as soon as the current scheduler command is completed bit 4 : START_ENABLE – Enables the module to process a global start command. Bit 5 : STOP_ENABLE – Enables the module to process a global stop command.
01	SCHED_STATUS	bit 0 : RUN – Indicates the scheduler is running bit 1 : WAIT – Module received a global start command and is waiting for a fiducial clock to enter the run state.
02	RSVD	Readback is 0xdeadbeef
03	LVL1_CNT	Readback of the number of LVL1 received from GL1 Writing a '1' to bit 0 will clear the count
04	LVL1_ACCPT_CNT	Readback of the number of LVL1 accepts. Writing a '1' to bit 0 will clear the count
05	FEM_CNVRT_TIME	This timer sets the front end time to complete the conversion. Resolution is 1/9.4MHz, 32 bit length
06	FEM_ENDAT0_TIME	This timer sets the readout time Resolution is 1/9.4MHz, 32 bit length
07	FEM_ENDAT1_TIME	This timer sets the readout time Resolution is 1/9.4MHz, 32 bit length
08	USR_BITS	Sets the 3 user bits to the FEM
09	BCO_CNT_RST	Set to 1 to reset the Beam Crossing Counter to
0xA	SCH_RESET	Resets the current vGTM scheduler
0xB	L1_DELAY	Coarse delay in units of Beam Clock (9.4MHz) for the L1 (16bit value)

0xC	LOCAL_RUN	When the GTM is in local mode, use this register to start / stop the scheduler. 0=stop, 1=run
0xD	GL1_PERMIT	Permits GL1 triggers for the current vGTM when in local mode. Ignored when in Global Mode 0=GL1 disabled, 1=GL1 permitted
0xE	LT_TIMER	This timer gets started when the maximum number of queued events have been asserted (As defined in MAX_EVENT_QUEUE register). BUSY is asserted until this counter reaches 0.
0xF	GLOBLOCAL_MODE	Selects whether the vGTM subsystem starts and stops via the global start or individually 0 = local mode (individual start / stop capability) 1 = global mode (single global start /stop) Note that busy signals are not passed to GL1 in local mode so that different subsystems busies won't affect each others operation.
0x10	FINE_DLY00	Fine delay of GTM FEM Data. Delay is 1st half of 1st NB6L295 delay chip on FFO board Bits 8-0 : each delay step is 11ps
0x11	FINE_DLY01	Fine delay of GTM FEM Data. Delay is 2nd half of 1st NB6L295 delay chip on FFO board Bits 8-0 : each delay step is 11ps
0x12	FINE_DLY10	Fine delay of GTM FEM Data. Delay is 1st half of 2nd NB6L295 delay chip on FFO board Bits 8-0 : each delay step is 11ps
0x13	FINE_DLY11	Fine delay of GTM FEM Data. Delay is 2nd half of 2nd NB6L295 delay chip on FFO board Bits 8-0 : each delay step is 11ps
0x20	DCM_BUSY_MASK	The DCM busy mask tells the vGTM which busy inputs are enabled and connected.
0x21	FELIX_BUSY_MASK_ MINIPOD0	The FELIX busy mask selects which busy inputs are enabled and connected for this vGTM from miniPOD 0.
0x22	FELIX_BUSY_MASK_ MINIPOD1	The FELIX busy mask selects which busy inputs are enabled and connected for this vGTM from miniPOD 1.
0x23	FELIX_BUSY_MASK_ MINIPOD2	The FELIX busy mask selects which busy inputs are enabled and connected for this vGTM from miniPOD 2.
0x24	FELIX_BUSY_MASK_ MINIPOD3	The FELIX busy mask selects which busy inputs are enabled and connected for this vGTM from miniPOD 3.
0x25	FELIX_BUSY_MASK_ MINIPOD4	The FELIX busy mask selects which busy inputs are enabled and connected for this vGTM from miniPOD 4.
0x400-	SCH_MEM	Scheduler Memory (256 word) See section 2.1 for details

0x40FF		
0x1000-0x1FFF	COM_MEM	Command Memory (4K word) Provides 32 unique command groups (128 length each)

8.0 Interfacing with the GL1-GTM

Since the system includes an ARM processor running linux, connecting to it is similar to connecting to any linux based computer, one standard way is to use ssh.
> ssh root@ipaddress
the default username and password is root,root.

8.1 Register Reads & Writes

A simple command line interface is provided to program all the registers of the system and is called reg_rw. A unique GTM address space is provided for each of the virtual GTMs (0-13) as well as a global adresss space which is used for registers that affect all virtual GTMs (G) as well as GL1 and system level registers. The format of the command is as follows. See section 7.0 for a definition of the Register Map.

> ./reg_rw [G,0..13] [R/W] address data

Examples:
To write to the global address space of the GTM, to write the Global Start command,
> ./reg_rw G W 0 1

To write to virtual GTM0 to set the FEM user bits to the value 3
> ./reg_rw 0 W 8 3

To read the temperature sensor on the board
> ./reg_rw G R 12

8.2 System initialization

On power up, the GL1-GTM initialization has a few initialization steps
1. Program the external PLL. The Si5338 PLL provides a 108MHz reference clock for the GTX transceivers. This part needs to be programmed, and a c program called si5338 is provided.

```
> ./si5338/si5338
```

2. Reset the GTX transceivers. This is done by writing the global address space of the GTM and will reset the GTX of all the Front End links.

```
> ./gtm_rw G W 8 255
> ./gtm_rw G W 8 0
```

8.3 GTM Initialization

3. Reset the mode bit scheduler. This is done by writing the global address space of the GTM and will reset all virtual GTM mode bit schedulers

```
> ./reg_rw G W 5 1
> ./reg_rw G W 5 0
```

A script is provided which will perform all the above steps

```
> gtm_init
```

8.4 GTM Mode Bit Scheduler Interface

As defined in section 2.0 of the document the GTM mode bit scheduler consists of a command memory and a scheduler memory. Some command line tools have been provided to help load these memories and also start and stop the scheduler.

To load a command memory, use the load_cmd program, which takes as arguments the virtual GTM number, and command mode group that will be loaded. The program also expects a text file with 120 entries which are the mode bit values to be transmitted to the front end for that revolution.

```
> ./load_cmd vgtm# modegrp# command_file
```

For example, to load the command file called cmdbits_allzeros into vGTM 0, mode group 0 the entry would be as follows:

```
> ./load_cmd 0 0 cmdbits_allzeros
```

To load the scheduler memory, use the load_sch program, which takes as arguments the virtual GTM number and a text file with the scheduler commands (see section 2.1 for details)

```
> ./load_sch vgtm# scheduler_file
```

An example scheduler file is shown below which will cycle through the first 4 command mode groups at a rate of 1Hz each. The three fields for each line of the scheduler file are the retransmit bit, repeat value, and mode group number.

```
0 78333 0    # retransmit = 0, repeat value = 78333 (78333*9.4MHz=>1Hz), mode group 0
0 78333 1    # retransmit = 0, repeat value = 78333 (78333*9.4MHz=>1Hz), mode group 1
0 78333 2    # retransmit = 0, repeat value = 78333 (78333*9.4MHz=>1Hz), mode group 2
1 78333 3    # retransmit = 1, repeat value = 78333 (78333*9.4MHz=>1Hz), mode group 3
```

Once the command memory and scheduler memory have been loaded into memory, the scheduler can be started and stopped.

To start the scheduler

```
> ./glob_start
```

To stop the scheduler

```
> .glob_stop
```

To quickly test the GTM scheduler, a predefined command and scheduler pattern has been setup which will toggle through the first four mode bits.

```
> ./load_loopleds_gtm0    # loads gtm0 with a pattern at 1Hz
> ./load_loopleds_gtm1    # loads gtm1 with a pattern at 2Hz
> ./glob_start
```

8.5 GL1 Initialization

The GL1 subsystem operation is described in section 5. Before the GL1 subsystem is ready for operation a few parameters of the system need configuration.

The scaledown registers need to be loaded. A helper script is provided which loads values from a text file. The text file expect a file with 64 lines, where each line has the scaledown value for the 64 triggers available.

```
> ./load_scaledowns <scaledown file>
```

The trigger look up table also needs to be configured. A helper script is provided which loads the table from a text file. The text file expects a file with 8 lines, where each line has 2 values. The first value is the trigger lookup table address and the second value is the value loaded into that address.

```
> ./load_triglut <triglut file>
```

A script is provided which will perform all the above steps

> ./gl1_init

Once the parameters have been loaded, soft triggers can be generated to exercise the system

> ./gen_softlvl1 <val> where val is the input trigger vector

For each GL1 trigger generated a snapshot of all 192 counters are clocked into FIFO's along with the value of the beam crossing counter so that they can be transmitted to the jSEB interface for transmission to the jSEB for archival. A throttled version of the same information is also available to the ARM processor for slow feedback of the state of the counters. To read out the counter values from the ARM processor a helper program is provided.

> ./get_countstats

which will produce an output shown below

```
Trigger Event Available
Beam Counter     : 1710881892746
Beam Counter Diff: 5000000
Trg#  ScaleDown    State     Raw Trig Cnt    Live Trig Cnt    Scaled Trig Cnt
-------------------------------------------------------------------------------
 0:    4999999    Enabled   1016825000000   1016825000000          203365
 1:          1    Enabled              76              76              38
 2:          0    Enabled             102             102             102
 3:          0    Enabled               0               0               0
 4:          0    Enabled               0               0               0
 5:          0    Enabled               0               0               0
 6:          0    Enabled               0               0               0
 7:          0    Enabled               0               0               0
 8:          0   Disabled               0               0               0
 9:          0   Disabled               0               0               0
10:          0   Disabled               0               0               0
11:          0   Disabled               0               0               0
12:          0   Disabled               0               0               0
13:          0   Disabled               0               0               0
14:          0   Disabled               0               0               0
15:          0   Disabled               0               0               0
```